

Wildlife Conservation Research Network

A multi-site conservation platform for genetic lineage tracking, behavioral research, breeding program management, and population analytics.

Database: MySQL 8.0+ | Language: Python 3.10 / Tkinter | Primary Focus: SQL Database Management

1. Executive Summary

The Wildlife Conservation Research Network (WCRN) is a full-stack desktop application built to manage a collaborative network of wildlife sanctuary sites. The system supports the full operational lifecycle of conservation work — from admitting animals and tracking their genetic lineage, to managing inter-site breeding programs, logging behavioral field observations, coordinating veterinary care, and generating population-level analytics.

The architecture is deliberately database-first. All critical business logic — eligibility validation, atomic multi-table workflows, audit trails, and scheduled automation — lives inside MySQL through stored procedures, triggers, views, and events. Python/Tkinter provides a clean presentation layer, keeping a clear separation between application code and data logic. This design reflects how real conservation databases are built and demonstrates the full range of relational database competency.

2. Project Objectives

2.1 Primary Objective

Build a production-grade MySQL-backed conservation platform that demonstrates advanced database management skills. Every advanced SQL feature in the system — recursive CTEs, window functions, table partitioning, BEFORE INSERT triggers, stored procedures — exists because the conservation domain genuinely requires it, not as a demonstration bolted onto a simpler problem.

2.2 Secondary Objectives

- Manage a multi-facility sanctuary network with cross-site population tracking
- Track genetic lineage through self-referencing animal family trees
- Enforce breeding eligibility at the database layer using recursive pedigree analysis
- Capture behavioral observation time-series for per-animal trend analysis
- Coordinate inter-facility animal transfers with quarantine protocols
- Maintain complete audit trails across all operational state changes
- Deliver role-differentiated access for Admins, Veterinarians, Researchers, and Caretakers

3. System Overview

WCRN models a conservation authority that oversees multiple physical sanctuary sites. Each site contains habitat zones. Animals live in zones, belong to species, and are connected to one another through genetic parentage. Staff are assigned to facilities. Research projects span multiple sites and accumulate behavioral observation data over time. Breeding programs coordinate pair selection across the network with database-enforced genetic compatibility rules.

The platform serves four operational domains simultaneously:

Domain	Scope	Key Data Entities
Facility & Population	Multi-site sanctuary management, zone capacity, species distribution	facilities, zones, animals, species, transfers
Genetic & Breeding	Pedigree tracking, inbreeding detection, breeding program coordination	animals (self-ref), breeding_programs, breeding_pairs
Health & Veterinary	Clinical records, vital sign monitoring, medicine, treatment management	health_records, vital_signs, medicines, treatments, incidents
Research & Analytics	Behavioral field observations, research projects, population reporting	observations, research_projects, project_enrollments

4. Users & Role-Based Access Control

Four distinct roles are enforced at both the UI and database layers. Permissions are applied per module — read-only access is granted where a role needs visibility without write capability.

Role	Access Level	Modules & Permissions
Admin	Full system	All 14 modules — full read/write. User management, system config, trigger/event monitoring, audit log.
Veterinarian	Health & Clinical	Health records (rw), Vital signs (rw), Medicines (rw), Treatments (rw), Animals (read), Incidents (rw), Breeding (read), Reports (read).
Researcher	Research & Data	Observations (rw), Research projects (rw), Animals (read), Species (read), Pedigree explorer (read), Behavioral analytics (read).
Caretaker	Daily Operations	Animals (read), Feeding schedules (rw), Zones (read), Tasks (rw), Incidents (report only).

5. Database Schema

5.1 Entity Catalog — 16 Tables

The schema is organised into four logical groups matching the operational domains. Every table has a defined purpose — no table exists solely for demonstration.

Group A — Facility & Population

Table	Primary Key	Purpose & Key Columns
facilities	facility_id (PK)	Sanctuary sites. name, location, country, established_date, total_area_sqkm, director_id (FK staff).
zones	zone_id (PK)	Habitat areas within a facility. facility_id (FK), habitat_type ENUM, area_sqkm, capacity, lat, lng. Zones have geospatial centroid coordinates.
species	species_id (PK)	Master species registry. common_name, scientific_name, iucn_status ENUM (Extinct→Least Concern), min_breeding_age, gestation_days, min_genetic_diversity_threshold, typical_weight_range_kg (JSON).
animals	animal_id (PK)	Core entity with self-referencing parentage. name, species_id (FK), father_id (FK animals), mother_id (FK animals), sex ENUM, dob, weight_kg, current_zone_id (FK), health_status ENUM, is_in_quarantine BOOL, arrived_on DATE.
staff	staff_id (PK)	Personnel across all facilities. facility_id (FK), name, role ENUM (Admin/Veterinarian/Researcher/Caretaker/Ranger), email, phone, speciality, is_active.
feeding_schedules	schedule_id (PK)	Diet plans per animal. animal_id (FK), diet_type, food_items TEXT, quantity_kg, times_per_day, notes.

Group B — Genetic Lineage & Breeding

Table	Primary Key	Purpose & Key Columns
breeding_programs	program_id (PK)	A coordinated effort for one species. program_name, species_id (FK), target_population_size, min_genetic_diversity, start_date, end_date, lead_vet_id (FK staff), status ENUM.
breeding_pairs	pair_id (PK)	A proposed or active breeding pair within a program. program_id (FK), sire_id (FK animals), dam_id (FK animals), proposed_date, approved_date, approving_vet_id, outcome ENUM, offspring_count. BEFORE INSERT trigger validates genetic compatibility.
transfers	transfer_id (PK)	Inter-facility animal movements. animal_id (FK), from_facility_id (FK), to_facility_id (FK), requested_by (FK staff), approved_by (FK staff), transfer_date, quarantine_start, quarantine_end_expected, transfer_reason, status ENUM.

Group C — Health & Veterinary

Table	Primary Key	Purpose & Key Columns
health_records	record_id (PK)	Clinical encounter log. animal_id (FK), attending_vet_id (FK staff), encounter_date, encounter_type ENUM (Routine/Emergency/Followup), diagnosis TEXT, prognosis ENUM, requires_isolation BOOL, cleared_date.
vital_signs	vital_id (PK)	Structured measurement time-series. record_id (FK health_records), measured_at DATETIME, weight_kg, temp_celsius, heart_rate_bpm, respiratory_rate, blood_glucose. Proper clinical data — not a notes field.
medicines	medicine_id (PK)	Inventory. name, type, unit, stock_qty, min_stock, expiry_date, supplier, contraindicated_species JSON (array of species_ids). The JSON column enables trigger-level contraindication checking.
treatments	treatment_id (PK)	Links a medicine to a health record. record_id (FK), medicine_id (FK), administered_by (FK staff), administered_at DATETIME, dose_amount, dose_unit, treatment_notes.
incidents	incident_id (PK)	Injuries, outbreaks, escapes, conflicts. incident_type ENUM, occurred_at DATETIME, reported_by (FK staff), severity ENUM (Low/Medium/High/Critical), description, resolution_notes, resolved_at.
incident_animals	(incident_id, animal_id)	Many-to-many junction. Links animals to incidents. role ENUM (Primary/Secondary/Witness). AFTER

		INSERT trigger flags involved animals for health assessment.
--	--	--

Group D — Research & System

Table	Primary Key	Purpose & Key Columns
research_projects	project_id (PK)	Funded research initiatives. title, lead_researcher_id (FK staff), species_id (FK), start_date, end_date, budget, funding_source, abstract TEXT, status ENUM.
project_enrollments	(project_id, animal_id)	Junction linking animals to projects with enrollment_date and notes. Observations reference projects through enrolled animals.
observations	obs_id (PK)	Behavioral field observation time-series. animal_id (FK), observer_id (FK staff), project_id (FK), observed_at DATETIME, behavior_type ENUM (Feeding/Social/Aggressive/Resting/Mating/Foraging/Distressed), duration_minutes, intensity ENUM (Low/Medium/High), location_in_zone VARCHAR, notes TEXT. PARTITIONED BY RANGE(YEAR(observed_at)).
audit_log	log_id (PK)	Immutable change history. table_name, operation ENUM (INSERT/UPDATE/DELETE), record_id, performed_by (FK users), performed_at DATETIME, old_values JSON, new_values JSON. Auto-populated by 9 triggers.
users	user_id (PK)	Login accounts. username, password_hash (SHA-256), role ENUM, staff_id (FK), is_active, last_login DATETIME.

6. Key Relationships & Constraints

6.1 Self-Referencing Parentage (The Signature Feature)

The animals table carries two optional self-referencing foreign keys:

animals.father_id	References animals.animal_id ON DELETE SET NULL. Nullable — wild-caught animals have no known lineage.
animals.mother_id	References animals.animal_id ON DELETE SET NULL. Same nullability rules.

These two columns unlock the most architecturally significant queries in the project — recursive CTEs that walk the pedigree tree upward any number of generations. They are the database foundation for all breeding compatibility logic.

6.2 Referential Integrity Rules

Relationship	ON DELETE Rule	Rationale
animals → species	RESTRICT	Cannot delete a species while animals of that species exist. Protects data integrity.
animals → zones	SET NULL	Animal record is preserved if a zone is decommissioned. Zone assignment becomes unset.
animals → animals (parents)	SET NULL	Losing a parent record does not lose the child animal — parentage becomes unknown.
health_records → animals	CASCADE	Clinical history belongs to the animal. If an animal is archived, records go with it.
vital_signs → health_records	CASCADE	Measurements are part of the encounter record. No orphan vitals.
treatments → health_records	CASCADE	Treatments are attached to encounters, not directly to animals.
treatments → medicines	RESTRICT	Cannot delete a medicine with active treatment records linked to it.
observations → animals	CASCADE	Observations belong to the animal subject.
breeding_pairs → animals	RESTRICT	Cannot remove an animal enrolled in an active breeding pair.
transfers → facilities	RESTRICT	Cannot delete a facility with transfer records — historical moves are permanent.

6.3 CHECK Constraints

animals.dob	dob <= CURDATE() — animals cannot have a future birth date
animals.weight_kg	weight_kg > 0 AND weight_kg < 30000
vital_signs.temp_celsius	temp_celsius BETWEEN 20 AND 45 — biologically plausible range
vital_signs.heart_rate_bpm	heart_rate_bpm BETWEEN 10 AND 400
zones.capacity	capacity > 0
zones.area_sqkm	area_sqkm > 0
medicines.stock_qty	stock_qty >= 0
breeding_pairs	sire_id <> dam_id — an animal cannot be paired with itself
observations.duration_minutes	duration_minutes > 0
transfers	from_facility_id <> to_facility_id — no same-site transfers

7. Advanced SQL Features

This section documents every advanced SQL construct in the system. Each feature is domain-native — it solves a real conservation problem that cannot be solved without that construct.

7.1 Recursive CTE — Genetic Pedigree Analysis

When a breeding pair is proposed, the system must determine whether the two animals share ancestors within the last three generations. This is impossible without a recursive query — there is no fixed-depth JOIN that handles arbitrary family trees.

Query: pedigree_ancestors	WITH RECURSIVE cte AS: starting at an animal, walk father_id and mother_id upward. Returns all ancestors with their generation depth. Used by the breeding eligibility trigger and the Pedigree Explorer UI.
Query: inbreeding_check	Run pedigree_ancestors for both proposed sire and dam. Intersect results within 3 generations. If any shared ancestor is found, the pair is flagged as incompatible.
Query: full_lineage_tree	Bidirectional traversal — walk both upward (ancestors) and downward (descendants) from a given animal. Drives the interactive family tree panel.
Query: lineage_depth	Aggregate the recursive result to find the deepest known ancestor and total known relatives per animal. Used in the species genetic diversity score.

7.2 Window Functions — Behavioral & Clinical Analytics

The observations table accumulates hundreds of rows per animal. Window functions are the only practical way to answer trend questions over this time-series data.

Function	Applied To	Answers
AVG() OVER (PARTITION BY animal_id ORDER BY observed_at ROWS 6 PRECEDING)	observations	7-day rolling average intensity per behavior type — is this animal's stress trending up?
RANK() OVER (PARTITION BY species_id ORDER BY aggression_score DESC)	observations aggregated by animal	Which animals rank highest for aggressive behavior within their species cohort?
LAG(weight_kg, 1) OVER (PARTITION BY animal_id ORDER BY measured_at)	vital_signs	Week-over-week weight change per animal — detect abnormal loss or gain.
COUNT() OVER (PARTITION BY zone_id ORDER BY arrived_on)	animals	Running population count per zone over time — capacity trend analysis.
NTILE(4) OVER (PARTITION BY species_id ORDER BY obs_count)	observation counts per animal	Quartile rank of observation coverage — identifies under-monitored animals.
STDDEV() OVER (PARTITION BY animal_id ORDER BY observed_at ROWS 29 PRECEDING)	vital_signs	30-day standard deviation of heart rate — flags clinical anomalies.

7.3 Table Partitioning — observations

The observations table is the largest in the system. A multi-site network logging daily behavioral data generates tens of thousands of rows per year. The table is range-partitioned by observation year so that date-scoped queries only scan the relevant partition.

Partition strategy	PARTITION BY RANGE (YEAR(observed_at)) — one partition per year from 2020 onwards, plus a MAXVALUE catch-all.
Query benefit	Queries filtered by date range (e.g. last 90 days) scan only 1–2 partitions instead of the full table. EXPLAIN PARTITIONS makes this visible in the UI.
UI demonstration	The Behavioral Analytics panel includes an EXPLAIN toggle that shows partition pruning in action — before and after comparison with row counts per partition scanned.
Partition maintenance	A MySQL EVENT runs on January 1 each year to add the next year's partition automatically.

7.4 Stored Procedures

Procedure	Tables Modified	Purpose
sp_admit_animal()	animals, zones, feeding_schedules, audit_log	Atomically registers a new animal: inserts the animal record, assigns to zone (checks capacity), creates a default feeding schedule, logs to audit_log. Rolls back entirely if zone is at capacity.
sp_transfer_animal()	animals, zones, transfers, audit_log (×2)	Inter-facility move protocol. Updates animal zone, inserts transfer record with quarantine dates, decrements source zone count, increments destination zone count. All 6 writes in one TRANSACTION with SAVEPOINT before each write — partial rollback on any failure.
sp_approve_breeding_pair()	breeding_pairs, audit_log	Sets pair status to Approved, records approving vet and date. Checks that the program is still active and neither animal's health has deteriorated since proposal. Raises SIGNAL SQLSTATE on failure.
sp_administer_treatment()	treatments, medicines, audit_log	Inserts treatment record and decrements medicine stock atomically. SAVEPOINT before stock update — if stock would go negative, rolls back to SAVEPOINT and raises an error without losing the treatment record.
sp_close_incident()	incidents, incident_animals, health_records, audit_log	Marks incident as resolved, clears quarantine flags on all involved animals not otherwise restricted, creates a follow-up health record for each. Multi-table write in a single COMMIT.
sp_generate_population_report()	read-only across 6 tables	Aggregates species count, health distribution, zone occupancy, and genetic diversity score per facility. Returns a result set consumed directly by the Reports tab.

7.5 Triggers — 9 Active Triggers

Trigger	Event	Action
trg_breeding_pair_eligibility	BEFORE INSERT on breeding_pairs	Runs recursive pedigree CTE. Calls SIGNAL SQLSTATE if: (a) shared ancestor within 3 generations found, (b) either animal is not Healthy, (c) either animal is below species min_breeding_age, or (d) species mismatch with program.
trg_quarantine_on_incident	AFTER INSERT on incident_animals	Sets is_in_quarantine = TRUE on all animals linked to a Critical or High severity incident. Inserts audit_log row per affected animal.
trg_medicine_contraindication	BEFORE INSERT on treatments	Reads medicines.contraindicated_species JSON. If the animal's species_id appears in the array, raises SIGNAL SQLSTATE with a descriptive message.
trg_health_status_change_log	AFTER UPDATE on animals (health_status)	Inserts a health_records row automatically when health_status changes — creates a system-generated encounter entry so no change goes unrecorded.
trg_medicine_low_stock	AFTER UPDATE on medicines	When stock_qty crosses below min_stock threshold, inserts an alert row into audit_log with event type LOW_STOCK.
trg_transfer_zone_counts	AFTER UPDATE on animals (current_zone_id)	Maintains a denormalised zone.current_count column in sync with animal moves — so zone capacity checks are O(1) reads.
trg_new_animal_audit	AFTER INSERT on animals	Logs the full new animal record to audit_log as a JSON snapshot.
trg_treatment_stock_deduct	AFTER INSERT on treatments	Decrements medicines.stock_qty by dose_amount. If stock would go negative, triggers SIGNAL (belt-and-suspenders alongside the stored procedure check).
trg_incident_health_flag	AFTER UPDATE on incidents (severity)	If severity is escalated to Critical, auto-sets health_status to Critical on all involved animals.

7.6 Views — 7 Named Views

View	Joins / Aggregations	Used By
vw_animal_profiles	animals JOIN species JOIN zones JOIN facilities JOIN staff (caretaker)	Animals panel — single query for the full animal roster with location and assignment
vw_sick_animals	animals JOIN health_records JOIN staff (vet) JOIN facilities — filtered to health_status IN (Sick, Critical)	Dashboard metric card + Veterinarian landing page
vw_medicine_alerts	medicines WHERE stock_qty <= min_stock — includes expiry_date < NOW() + 30 DAYS	Dashboard alert strip + Medicines panel
vw_active_treatments	treatments JOIN health_records JOIN animals JOIN medicines JOIN staff	Veterinarian workload panel — what treatments are in progress
vw_zone_occupancy	zones JOIN facilities — LEFT JOIN COUNT(animals) — shows current_count vs capacity and a derived occupancy_pct	Network Map panel + Zone management
vw_breeding_program_status	breeding_programs JOIN species JOIN COUNT(pairs) JOIN COUNT(offspring)	Breeding Program Manager — program dashboard row
vw_species_network_summary	GROUP BY species across all facilities — total count, healthy count, in-breeding-program count, avg_weight, latest_observation_date	Reports tab — population overview

7.7 Indexes

Index	Columns	Rationale
idx_animals_species_health	animals(species_id, health_status)	Species + health filter — used by vw_sick_animals and the Dashboard metric queries.
idx_animals_zone	animals(current_zone_id)	Zone occupancy count queries — supports trg_transfer_zone_counts.
idx_animals_parents	animals(father_id), animals(mother_id)	Two separate indexes. Recursive CTE ancestor walks follow FK paths — these make each recursive step a single index lookup.
idx_observations_animal_date	observations(animal_id, observed_at)	Covers every window function query partitioned by animal over a date range.
idx_observations_project	observations(project_id, observed_at)	Research project analytics — all observations for a project in date order.

idx_health_records_animal_date	health_records(animal_id, encounter_date)	Per-animal clinical history retrieval.
idx_vital_signs_record_time	vital_signs(record_id, measured_at)	Time-ordered vital sign queries per encounter.
idx_transfers_animal	transfers(animal_id, transfer_date)	Animal movement history — chronological transfer log.
FULLTEXT idx_obs_notes	observations(notes)	Full-text search across field observation notes from the Research Portal.

7.8 Transactions & SAVEPOINT Usage

Every multi-table write in the system is wrapped in an explicit transaction. The following scenarios demonstrate the full transaction feature set:

Animal admission	BEGIN → SAVEPOINT sp1 → INSERT animal → SAVEPOINT sp2 → assign zone (check capacity, rollback to sp1 if over) → INSERT feeding_schedule → COMMIT
Inter-facility transfer	BEGIN → SAVEPOINT sp1 → UPDATE animal zone → SAVEPOINT sp2 → INSERT transfer record → SAVEPOINT sp3 → UPDATE zone counts → COMMIT. Any failure rolls back to the appropriate SAVEPOINT.
Batch treatment logging	BEGIN → loop: SAVEPOINT sp_N before each INSERT treatment + UPDATE stock → on medicine error: ROLLBACK TO sp_N, skip, continue → COMMIT surviving treatments
Incident closure	BEGIN → UPDATE incident status → loop affected animals: SAVEPOINT sp_N → clear quarantine + INSERT health_record → COMMIT all on success
Rollback demonstration	The Transactions panel includes a live demo mode — executes a multi-step scenario and intentionally triggers a rollback at step 3, showing that steps 1 and 2 are unwound completely.

7.9 MySQL Events — Scheduled Automation

evt_partition_maintenance	Runs annually on January 1. Adds the next year's partition to the observations table using ALTER TABLE ... ADD PARTITION.
evt_weekly_health_sweep	Runs every Monday at 06:00. Queries animals with no health_record in the last 90 days and inserts an overdue_checkup alert into audit_log.
evt_expired_medicine_flag	Runs daily at midnight. Updates medicines.status to Expired where expiry_date < CURDATE() and status != Expired.

8. Application Modules

The application presents 14 panels organised by operational domain. Role-based tab filtering ensures each user sees only the panels relevant to their function.

Module	Roles	Core SQL
Dashboard	All	vw_sick_animals, vw_medicine_alerts, zone occupancy aggregates — 4 live metric cards on login
Animals	All (rw: Admin, Vet, Caretaker)	vw_animal_profiles, sp_admit_animal, sp_transfer_animal, LIKE search across name/species/facility
Pedigree Explorer	Admin, Researcher	Recursive CTE — interactive family tree, inbreeding score, ancestor table by generation depth
Breeding Programs	Admin, Vet	breeding_programs + breeding_pairs CRUD, sp_approve_breeding_pair, trg_breeding_pair_eligibility feedback
Behavioral Analytics	Admin, Researcher	Window function queries over observations — rolling averages, ranking, anomaly detection charts
Research Projects	Admin, Researcher	research_projects CRUD, project_enrollments, observation export, FULLTEXT search on obs notes
Observations	Researcher	INSERT observations with behavior type and intensity, EXPLAIN PARTITIONS viewer, date-range filter
Health Records	Admin, Vet	health_records + vital_signs combined view, LAG-based vital trend, clinical timeline
Medicines	Admin, Vet	vw_medicine_alerts, stock management, contraindication editor (JSON), expiry calendar
Treatments	Admin, Vet	sp_administer_treatment, vw_active_treatments, per-medicine treatment history
Incidents	Admin, Vet (report: Caretaker)	incidents + incident_animals, trg_quarantine_on_incident result display, sp_close_incident
Network Map	Admin	vw_zone_occupancy across all facilities, transfer history, sp_transfer_animal, quarantine roster
Reports	All (read)	sp_generate_population_report, window function analytics, CSV export, EXPLAIN viewer
System (Admin only)	Admin	Trigger status, Event status, audit_log viewer, Users CRUD, DB stats

9. Security Implementation

Password hashing	All passwords stored as SHA-256 hashes. No plaintext anywhere in the database or application code. Comparison is always hash-to-hash.
Parameterised queries	Every SQL statement uses %s placeholders via mysql-connector-python. String concatenation in SQL is never used — SQL injection is structurally impossible.
Environment credentials	DB_CONFIG reads from environment variables or a local .env file. No credentials are hardcoded in source files.
Role enforcement	Tab visibility is filtered at login. DB-layer views and stored procedures further restrict what each role can read or write — bypassing the UI does not bypass permissions.
Audit log immutability	The audit_log table has INSERT-only trigger population. No UPDATE or DELETE is permitted on audit_log — it is append-only by design.
Soft archive	Animals are never hard-deleted. A is_archived flag moves them out of active views while preserving all linked clinical, observation, and breeding history.
Session management	Login creates an in-memory session dict. Logout destroys it. No persistent session tokens stored in the database.

10. Deliverables

10.1 Code

1. wcrn.py — main Python/Tkinter application (~4,500+ lines across modular panel classes)
2. schema.sql — complete DDL: all 16 CREATE TABLE statements with constraints, 9 indexes, 7 views, 6 stored procedures, 9 triggers, 3 events, and partition definitions
3. seed_data.sql — realistic dataset: 4 facilities, 12 zones, 8 species, 60+ animals with linked lineage, 200+ observations, 30+ health records, 15+ treatments
4. requirements.txt — Python dependencies (mysql-connector-python, tkinter stdlib)

10.2 Documentation

5. This Project Scope Document
6. ER Diagram — full entity-relationship diagram including self-referencing parentage, partitioned table notation, and JSON column callouts
7. SQL Feature Reference — annotated query examples for the recursive CTE, window functions, and partition EXPLAIN output
8. README.md — setup guide, credential configuration, seed data instructions

11. Implementation Order

Ordered by dependency and foundational importance — each phase builds on the one before.

Phase	Tasks	Produces
1 — Foundation	Create facilities, species, zones, staff, animals tables with all constraints and CHECK clauses. Add father_id / mother_id self-referencing FKs. Seed 4 facilities and 8 species.	Working schema, basic animal CRUD
2 — Clinical	Add health_records, vital_signs, medicines, treatments, incidents, incident_animals. Implement trg_medicine_contraindication and trg_treatment_stock_deduct triggers.	Full veterinary workflow
3 — Lineage	Add breeding_programs, breeding_pairs. Write the recursive pedigree CTE. Implement trg_breeding_pair_eligibility. Build Pedigree Explorer panel.	Recursive CTE, BEFORE INSERT trigger
4 — Research	Add observations table with RANGE partitioning. Add research_projects, project_enrollments. Implement FULLTEXT index. Build Behavioral Analytics window function queries.	Partitioning, window functions, full-text
5 — Network	Add transfers table. Implement sp_transfer_animal with full SAVEPOINT transaction. Build Network Map panel. Add trg_transfer_zone_counts.	Stored procedures, multi-SAVEPOINT transactions
6 — Automation	Add all 3 MySQL EVENTS. Implement remaining triggers. Build audit_log population across all 9 triggers. Add evt_weekly_health_sweep.	Events, complete audit trail
7 — Views & Reports	Create all 7 views. Build sp_generate_population_report. Build Reports tab with window function charts and CSV export. Add EXPLAIN PARTITIONS viewer.	Views, reporting, partition demo
8 — Polish	Dashboard metric cards. FULLTEXT search in Research Portal. Role-based tab filtering. Login with SHA-256 hashing. System admin panel.	Complete, demo-ready application

12. Success Criteria

The project is complete and ready for submission when all of the following can be demonstrated live:

- Recursive CTE correctly identifies shared ancestors and blocks an inbreeding-risk breeding pair at the database layer
- At least four distinct window functions (rolling AVG, RANK, LAG, STDDEV) running over the observations or vital_signs time-series
- EXPLAIN PARTITIONS output visibly shows partition pruning on a date-filtered observations query
- sp_transfer_animal completes a 6-write atomic transfer and visibly rolls back when destination zone is at capacity
- All 9 triggers are verifiable in information_schema.TRIGGERS and fire correctly under their conditions
- All 3 MySQL EVENTS are verifiable in information_schema.EVENTS and show a LAST_EXECUTED timestamp
- 7 views return correct results and are used by their respective application panels
- audit_log captures every INSERT, UPDATE, and DELETE across all operational tables with the correct user and timestamp
- Contraindication trigger blocks a treatment insertion with a descriptive SQLSTATE error surfaced in the UI
- All passwords in the users table are stored as 64-character SHA-256 hex strings — zero plaintext
- CSV export produces a correctly formatted file from at least two different report types
- Seed data is rich enough to make all analytics meaningful — at minimum 60 animals across 4 facilities with complete lineage chains